# Differentiation, Accessibility, and Classroom Support

Guidance for beginner and intermediate variants, readability, pacing, and learner support.

Reference material for lesson planning and course-content development.

# Contents

# 1. Adapting without changing the lesson goal

Differentiation changes the amount of support, the complexity of the context, or the independence expected from the learner. It should not quietly replace the objective with a different topic. If the lesson is about Python loops, an advanced variant may combine a loop with a list or debugging task, but it should not become a lesson on asynchronous programming.

## Three levers

- Support: hints, examples, traces, checklists.
- Complexity: number of steps, amount of information, familiarity of context.
- Independence: how much of the process the learner must choose.

Keep at least one common task across levels so the teacher can compare understanding of the core concept.

## 2. Beginner support

Beginner materials should make hidden steps visible. Define new vocabulary, show short examples, and keep instructions focused. Learners may need a worked example and a partially completed practice task before independent work.

- Use descriptive names and labels.
- Show one change at a time.
- Provide small hints rather than full solutions.
- Use checks for understanding before moving on.

Do not mistake 'beginner' for 'childish'. University learners still benefit from direct, respectful language and realistic contexts.

## 3. Intermediate challenge

Intermediate learners can handle less familiar contexts and can combine ideas that were already taught. Challenge should come from decisions, diagnosis, or transfer rather than from obscure terminology.

- Ask which method is better and why.
- Use a broken example with more than one plausible cause.
- Ask learners to compare two solutions.
- Reduce the number of prompts.

When an intermediate version becomes longer, check that the extra material still fits the stated duration.

## 4. Accessibility in text and layout

Learning material should remain understandable when copied into plain text. Do not rely on color alone for correct/incorrect status, sequence, or category. Use labels and wording.

- Use headings that describe the section.
- Keep paragraphs reasonably short.
- Separate code, formulas, examples, and instructions.
- Use alt text when images are part of the final delivery environment.
- Avoid tiny text or dense tables.

If a table becomes too wide, split it rather than shrinking the text until it is difficult to read.

# 5. Supporting multilingual learners

Technical instruction can be difficult when the learner is simultaneously learning the subject and the language used to teach it. Keep important terminology consistent. Do not use several synonyms for the same technical concept just to make the writing sound varied.

## Useful support

- Define the term in plain English.
- Give a short example.
- Use the same term again in practice.
- Avoid idioms in instructions.

A bilingual glossary can help when available, but the code, formulas, API fields, and official technical terms should remain accurate.

## 6. Time-limited learners

If a learner has less time than the full lesson, preserve the core explanation, one worked example, one practice task, and one check. Remove enrichment before removing the main evidence of learning.

| Time available | Keep |
|---|---|
| 20 minutes | Core idea, one example, one practice check |
| 35 minutes | Core idea, example, guided practice, quick check |
| 60 minutes | Full lesson with guided and independent practice |

A shorter version should say what was omitted so the learner or teacher knows what remains.

## 7. Remote and self-paced use

Self-paced material needs clearer instructions because there may be no teacher present to clarify the next step. Every activity should say what to do, what to produce, and how to check whether the result is reasonable.

- Include expected output for setup steps.
- Keep required files and commands explicit.
- Provide a short troubleshooting note for common setup problems.
- Avoid activities that depend on unlisted classroom materials.

For a lesson pack used through an API, keep sections independently readable because the user may ask to revise or regenerate only one section.

## 8. Supporting learners who finish early

Extension work should deepen the same objective. It can ask for comparison, optimization, explanation, or application to a new context. It should not simply add more repetitive questions.

- Explain why one solution is clearer than another.
- Create a second example for a peer.
- Find and fix a less obvious error.
- Apply the same concept to a new dataset or scenario.

Extensions should be optional and clearly separated from core requirements.

## 9. Handling common learning difficulties

When several learners make the same mistake, treat it as information about the lesson. Re-explain the key step using a different representation. For loops, show a trace table. For equations, show both sides changing. For forces, draw one object at a time.

When one learner is stuck, ask what they think the current state is. This often reveals whether the problem is vocabulary, a missing prerequisite, or the new concept itself.

- Do not respond to every difficulty by adding more text.
- Use examples and intermediate states.
- Check prerequisite knowledge when the same error repeats.
- Give the learner a chance to try again after feedback.

## 10. Checking differentiated versions

Compare the beginner and intermediate versions side by side. The main objective should still match. The beginner version should have more support, while the intermediate version should require more independent reasoning.

### Review questions

- Did the key facts remain the same?
- Did the level change without changing the topic?
- Are the examples appropriate for the audience?
- Does the quiz match the version actually taught?
- Did simplification accidentally remove an important condition?

If a simplified version changes a rule, number, or scientific fact, it is not an acceptable simplification.